

Devbobs MiniTwit Report

Course Name: DevOps, Software Evolution and Software Maintenance, BSc
(Spring 2026)

Course Code: BSDSESM1KU
2499 words, counted with TexCount

Written by:

Marius Emil Holm	marho@itu.dk
Jonas Boll Esser	jones@itu.dk
Torkil á Torkilsheyggi	toat@itu.dk
Morten Friis Bønneland	mofb@itu.dk
Laura Lundø Sandberg	llsa@itu.dk

Contents

1	System's Perspective	3
1.1	Design and architecture of our ITU-MiniTwit systems.	3
1.2	Dependencies	4
1.3	Current state of system	5
1.3.1	Practices	5
1.3.2	Tooling	5
2	Process Perspective	7
2.1	CI/CD Pipelines	7
2.1.1	Build and test	9
2.1.2	CI/CD	9
2.1.3	Compile report	9
2.1.4	Release	9
2.1.5	SonarQube	10
2.2	Monitoring	10
2.2.1	Business monitoring	10
2.2.2	Application monitoring	11
2.2.3	Infrastructure monitoring	11
2.3	Logging	12
2.3.1	What we log	12
2.3.2	Aggregation of logs	12
2.4	Security hardening	15
2.5	Availability and scaling	15
2.5.1	Availability	15
2.5.2	Scaling	15
3	Reflection Perspective	17
3.1	Devops-style work	17
3.2	Biggest challenges during development	17
3.2.1	Refactoring and evolution	17
3.2.2	Logging	18
3.2.3	Test-server	18
4	Use of Generative AI	19
5	Collection of links to artifacts	19
A	Dependencies	20

All members have participated in the making and editing of this report.

1 System's Perspective

This section will give a description supported by illustrations of the design and architecture of our MiniTwit system, the dependencies and the current state of our system.

The application is based on Chirp - a C# application made in the course Analysis, Design and Software Architecture at ITU. We chose to base the project on Chirp as we all had a general knowledge about the codebase.

1.1 Design and architecture of our ITU-MiniTwit systems.

The deployment architecture of the MiniTwit system can be seen in figure 1. The architecture strives to achieve high-availability by having a swarm of five nodes. The system is deployed on DigitalOcean using Terraform and Docker swarm. A user can reach MiniTwit with the domain `Devbobs.tech` on port 433 or 80. The request will reach the Leader node and will then get load-balanced by Caddy between the four instances of MiniTwit running. The Leader node also handles the logging and monitoring stack, which consists of Grafana, Loki and Prometheus. Alloy exists on every node to push logs to Loki. MiniTwit is also available on port 8080 with HTTP. The request will in this case be load-balanced by the docker ingress routing mesh. The same goes for Grafana, which is available on port 3000 and Prometheus on 9090. These can be reached with the floating IP or any of the nodes IPs'. A PostgreSQL database is hosted on DigitalOcean. The architecture has a single point of failure, which will be explained in section 2.5.1.

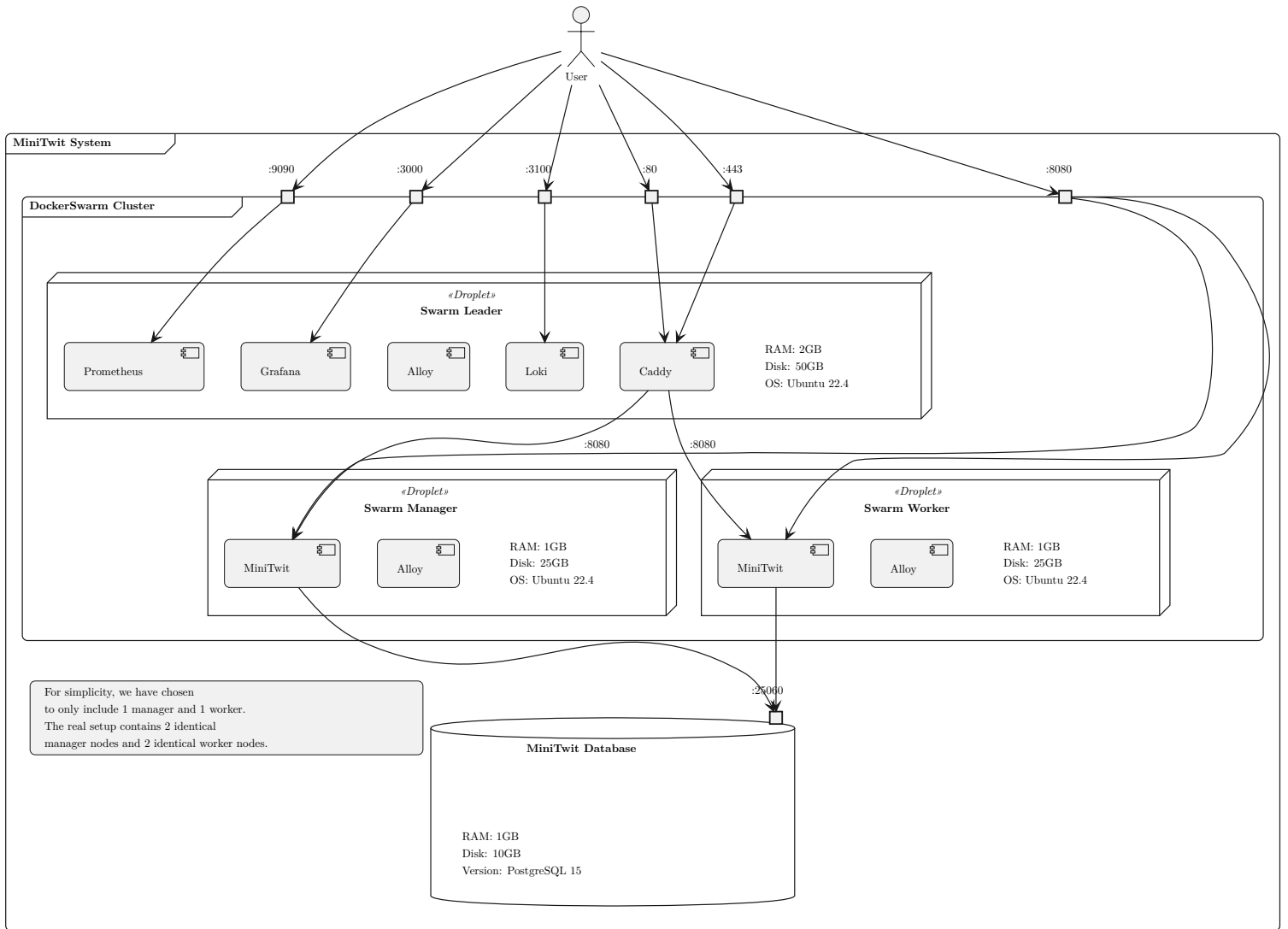


Figure 1: Deployment diagram of MiniTwit

1.2 Dependencies

MiniTwit has several dependencies in the form of Nuget, Docker and Github Action dependencies. They are listed on different levels of abstractions in appendix A. MiniTwit also has infrastructure and deployment dependencies, listed here.

- DigitalOcean - The platform used to host the MiniTwit system
- Terraform - The tool used to manage our infrastructure as code

- DockerHub - The platform used to store our docker images
- Docker - The tool used to package different parts of the MiniTwit system and run as services and containers on Droplets on DigitalOcean

In hindsight we have many dependencies, which could be reduced to ease later maintainance (e.g.: we have three different versions of the checkout actions in our GitHub Actions).

1.3 Current state of system

To keep software quality high, catch technical debt and later ease maintainability we follow several practices and use a set of different tooling.

1.3.1 Practices

The first practice is code reviews. No code gets pushed to main before it has been approved by at least one other group member. This is done through GitHub pull-requests. Combined with the use of GitHub issues, this gives a clear picture of why, what and how a feature have been implemented. The use of conventional commits like: `feat:`, `chore:`, `note:`, has been used to give a clear view of what each commit in the git history does.

1.3.2 Tooling

Tooling has been used to assist keeping quality high. This was especially useful as the project grew bigger. First, we use automatic testing, which has caught many problems. This is done in our continuous integration (CI) pipeline. The pipeline also runs a formatter and linter, which will be touched upon in 2.1.1

For static analysis we use SonarQube, which is used to analyse code quality, code smells and security issues. The current state of the system, according to SonarQube, is acceptable - SonarQube scores for our project are - Security: C, Reliability: A and Maintainability: A, as seen in Figure 2. The low score on security means we fail SonarQube's Quality Gate, which is an issue.

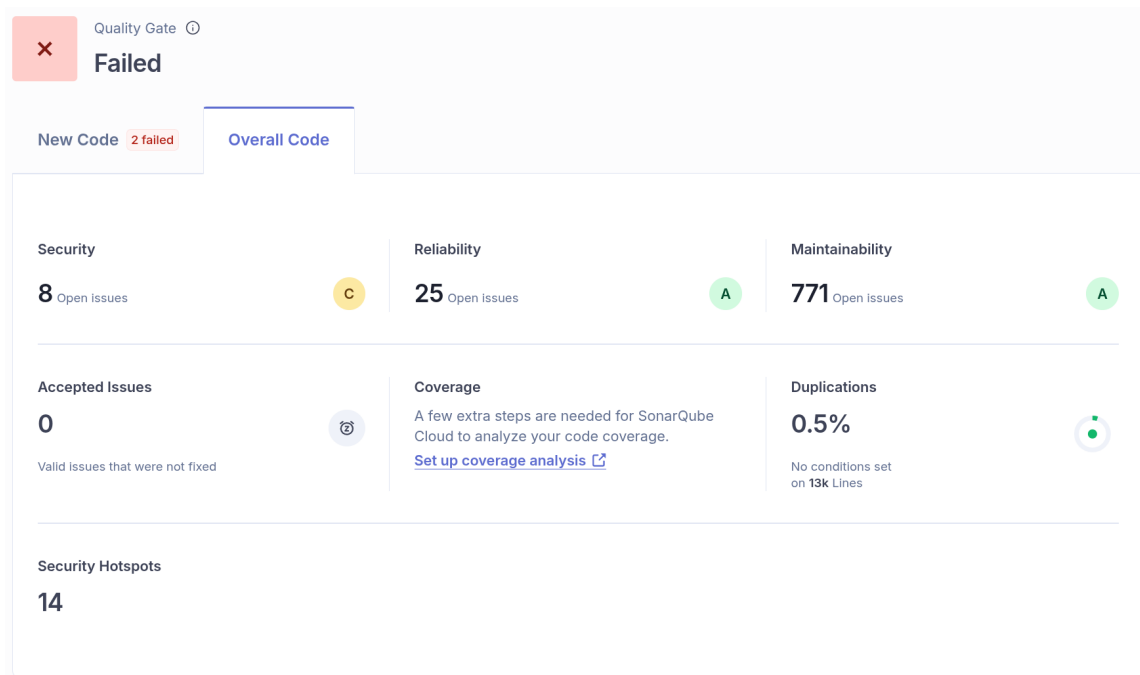


Figure 2: SonarQube Quality Gate Report

2 Process Perspective

2.1 CI/CD Pipelines

The CI/CD pipeline consists of three GitHub Action workflows, which can be seen in `/.github/workflows/`. All workflows are run on `ubuntu-24.04`. The first part of our CI pipeline (Build and test workflow) runs on pull-requests to main. The whole pipeline is run on pushes to main. The following section will give a description of each workflow and how a feature goes to production. This is also illustrated in figure 3.

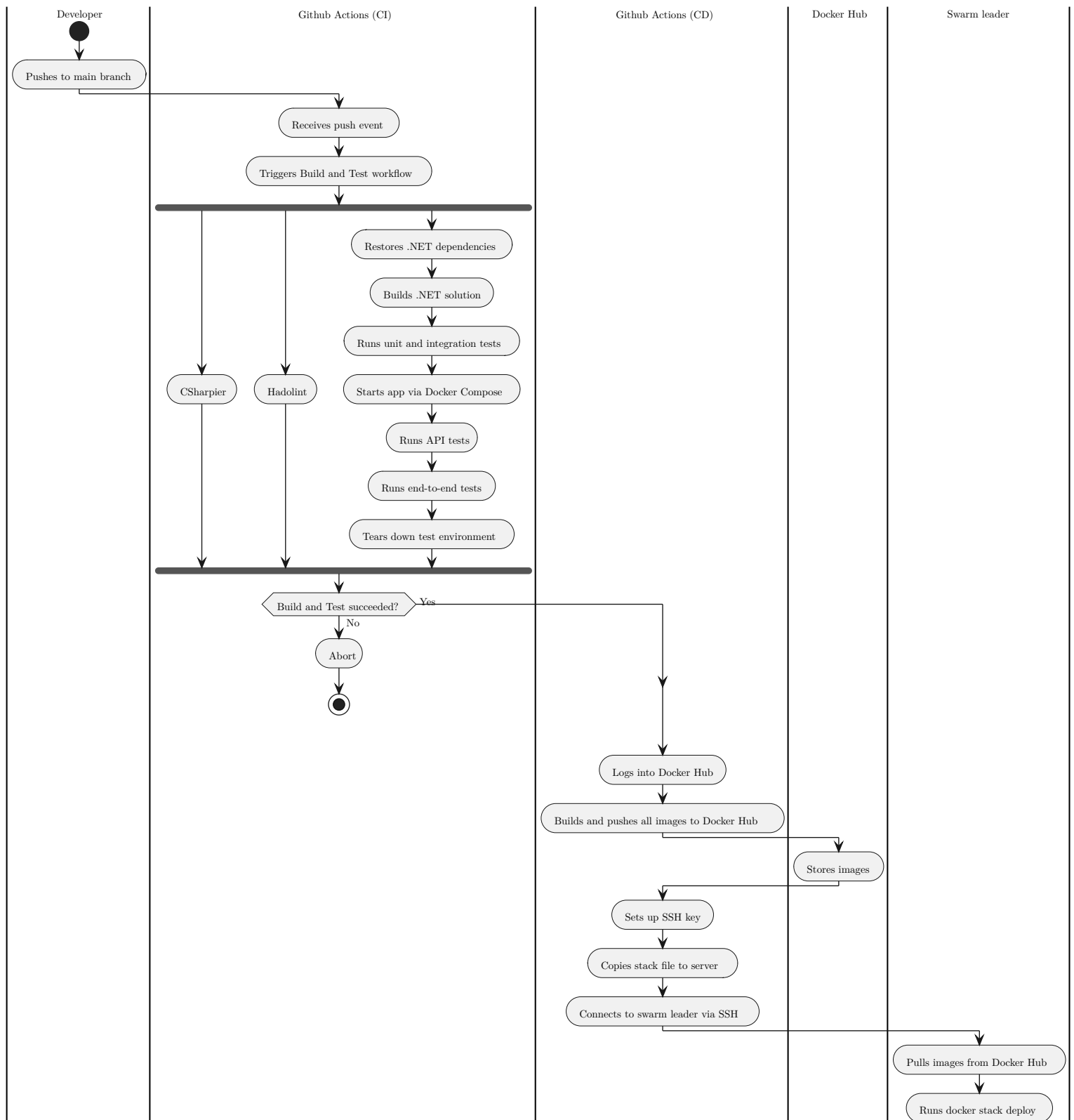


Figure 3: UML activity diagram of CI/CD pipeline. Swim lanes indicate responsible system or actor for each action.

2.1.1 Build and test

The first workflow (“*Build and Test*”), forms the continuous integration of our pipeline and consists of the following steps that run in parallel.

1. **Code formatter with Csharpier:** Checks the layout of the code. This helps ensure that the layout is alike throughout the codebase. It does not automatically apply layout changes, but reports it back on GitHub.
2. **Dockerfile linter with Hadolint:** Lints the MiniTwit dockerfile.
3. **Build the application:** Builds the applications and:
 - Runs unit and integration tests.
 - Runs tests on the API / simulator endpoint.
 - Runs E2E / Playwright tests.

2.1.2 CI/CD

The (“*CI/CD*”) workflow, forms our continuous deployment and runs on a successful *Build and Test* workflow and push to main. It consists of the following steps that run sequentially.

1. **Dockerhub:**
 - Github Actions builds and pushes the MiniTwit, Grafana, Alloy, Loki and Prometheus docker images to Dockerhub.
2. **SSH and Deploy**
 - SSH into the leader node.
 - Copy the stack file `/itu-minitwit/infrastructure/docker_swarm/stack/minitwit_stack.yml` to the node.
 - Deploy to the swarm.

2.1.3 Compile report

As part of our report writing, github actions compiles the report and stores it as an artifact.

2.1.4 Release

The “*release*” workflow runs on a successful CI/CD run. It builds and zips the MiniTwit application for Windows, Mac and Linux, zips the source code and this report. In hindsight our releases are not perfect. We automatically increased the version of each release which is why we ended up with version `0.0.259`, having no major or minor versions. The releases

did not contain any description, creating bad transparency. Lastly, instead of zipping our Minitwit application, we could have used our docker images to encapsulate each part of the system in releases. For now, these can be found at <https://hub.docker.com/u/powstdaws>.

2.1.5 SonarQube

SonarQube runs on pull requests as a step in our CI pipeline. It checks for security issues, code smells and code quality. SonarQube also gives a Quality Gate, that fails if the threshold for the pull-request is too low e.g. Security, Reliability and Maintainability. SonarQube is not blocking meaning we could still merge, even if SonarQube fails. It's more used as a reminder of better coding practices and error handling.

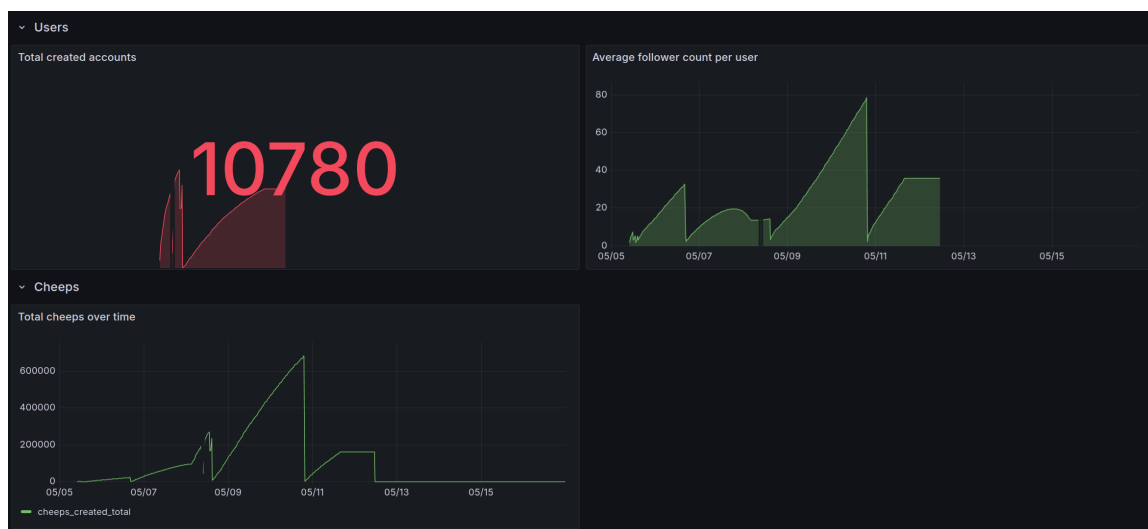
2.2 Monitoring

The system uses Prometheus and Grafana for monitoring. Prometheus scrapes the `/metrics` endpoint on each MiniTwit service instance. Grafana then queries Prometheus, which displays and visualises the data, see figure 5. Monitoring is done on the business level, application level, and infrastructure level.

2.2.1 Business monitoring

On the business level we monitor:

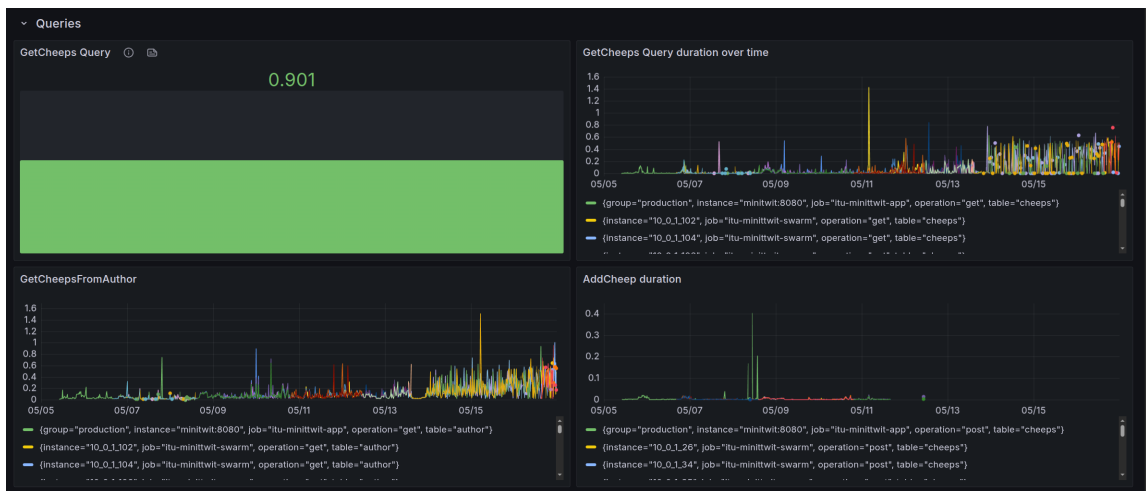
- Number of created accounts.
- Average follower count per user.
- Total number of cheeps over time.



2.2.2 Application monitoring

On the application level, we monitor the durations of different database queries that MiniTwit makes:

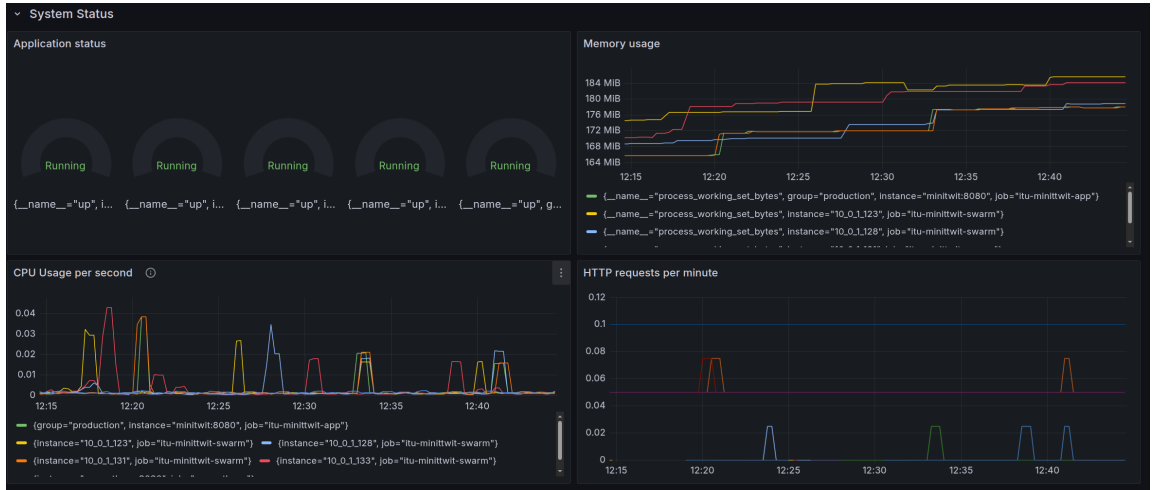
- GetCheeps - Gets cheeps for a given page.
- GetCheepsFromAuthor - Getting an Author's cheeps.
- AddCheep - Create a cheep.



2.2.3 Infrastructure monitoring

On the infrastructure level, we monitor:

- The status of nodes.
- Memory usage of nodes.
- CPU usage of nodes.
- HTTP requests per minute for each node.



2.3 Logging

The application uses a Grafana-loki-alloy stack to log all services. The logging setup can be seen in figure 5.

2.3.1 What we log

We currently log the logs from each docker service, as described in our `/infrastructure/docker_swarm/stack/minittwit_stack.yml` file. As will be discussed in 3.2.2, more comprehensive logging would have been beneficial.

2.3.2 Aggregation of logs

We use Alloy to collect logs from each node and forward them to Loki, see figure 4. The logs are displayed on grafana.

Service logs									
MiniTwit Logs					Grafana Logs				
labels	Time	Line	tsNs	id	labels	Time	Line	tsNs	id
{ "service": "m...	2026-05-18 12:09:32	Failed to determine th	17790989725828596	177909897258285	{ "service": "m...	2026-05-18 13:31:01	logger=tsdb.loki end;	177910386122913187	1779103861229131
{ "service": "m...	2026-05-18 12:09:32	warn: Microsoft.AspN	17790989725827988	177909897258279	{ "service": "m...	2026-05-18 13:31:01	logger=tsdb.loki end;	17791038612290638;	177910386122906
{ "service": "m...	2026-05-18 12:03:02	Failed to determine th	17790985820939203	177909858209392	{ "service": "m...	2026-05-18 13:31:01	logger=tsdb.loki end;	177910386117781948;	1779103861177819
{ "service": "m...	2026-05-18 12:03:02	warn: Microsoft.AspN	17790985820938176	177909858209381	{ "service": "m...	2026-05-18 13:31:01	logger=tsdb.loki end;	17791038611776736	177910386117767
{ "service": "m...	2026-05-18 12:01:14	Failed to determine th	17790984744262598	177909847442625	{ "service": "m...	2026-05-18 13:31:01	logger=tsdb.loki end;	177910386115148891;	1779103861151488
{ "service": "m...	2026-05-18 12:01:14	warn: Microsoft.AspN	17790984744261892	177909847442618	{ "service": "m...	2026-05-18 13:31:01	logger=tsdb.loki end;	177910386115140426	1779103861151404
Alloy logs					Promethues logs				
labels	Time	Line	tsNs	id	labels	Time	Line	tsNs	id
{ "service": "m...	2026-05-18 11:56:50	ts=2026-05-18T09:51	177909821086016081	177909821086016	{ "service": "m...	2026-05-18 13:00:03	time=2026-05-18T11:	177910200391699281	177910200391699
{ "service": "m...	2026-05-18 11:56:47	ts=2026-05-18T09:51	177909820750012871	177909820750012	{ "service": "m...	2026-05-18 13:00:03	time=2026-05-18T11:	17791020039056756	177910200390567
{ "service": "m...	2026-05-18 11:56:44	ts=2026-05-18T09:51	177909820476527121	177909820476527	{ "service": "m...	2026-05-18 13:00:03	time=2026-05-18T11:	17791020038938166;	177910200389381
{ "service": "m...	2026-05-18 11:56:44	ts=2026-05-18T09:51	17790982047592936	177909820475929	{ "service": "m...	2026-05-18 13:00:03	time=2026-05-18T11:	17791020038730046	177910200387300
{ "service": "m...	2026-05-18 11:56:44	ts=2026-05-18T09:51	17790982047576095	177909820475760	{ "service": "m...	2026-05-18 13:00:03	time=2026-05-18T11:	177910200334216118	177910200334216
{ "service": "m...	2026-05-18 11:56:44	ts=2026-05-18T09:51	17790982047570883	177909820475708	{ "service": "m...	2026-05-18 13:00:03	time=2026-05-18T11:	17791020033286579	177910200332865
Caddy logs					Loki Logs				
labels	Time	Line	tsNs	id	labels	Time	Line	tsNs	id
{ "service": "m...	2026-05-18 13:20:11	{ "level": "warn"; "ts": 177910321197844265	1779103211978442		{ "service": "m...	2026-05-18 13:31:01	level=info ts=2026-0	177910386129942731	177910386129942
{ "service": "m...	2026-05-18 13:20:11	{ "level": "warn"; "ts": 177910321197837378	1779103211978373		{ "service": "m...	2026-05-18 13:31:01	level=info ts=2026-0	177910386129941956	177910386129941
{ "service": "m...	2026-05-18 13:20:11	{ "level": "warn"; "ts": 177910321186392978	1779103211863929		{ "service": "m...	2026-05-18 13:31:01	level=info ts=2026-0	177910386129940731	177910386129940

Figure 4: Logging dashboard

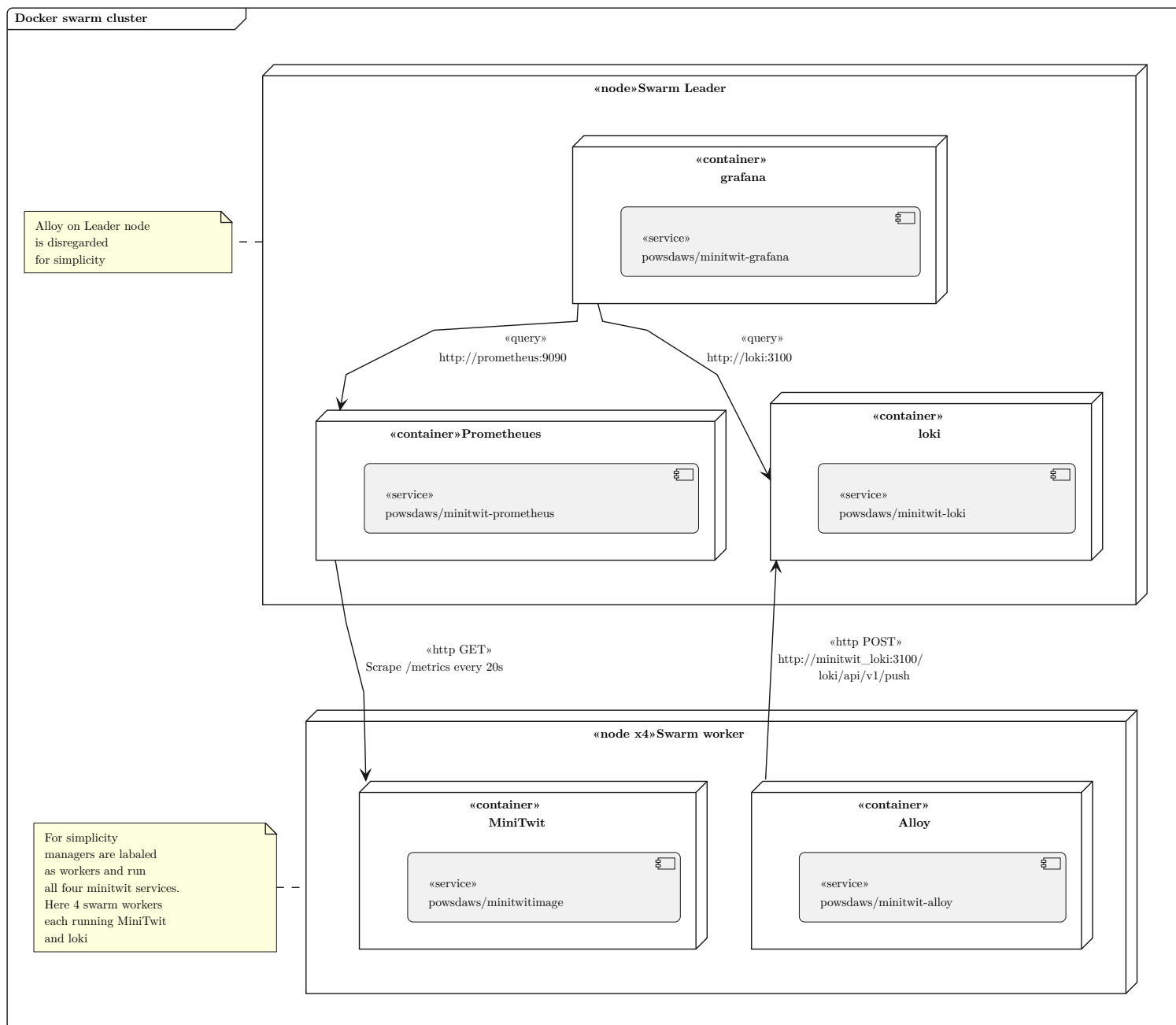


Figure 5: Deployment diagram of the monitoring and logging setup

2.4 Security hardening

To protect against supply line attacks, we would select docker hardened images from dhi.io to use for our dependencies; Prometheus, Alloy, Loki and Grafana. We would also go through our MiniTwit application and adjust it to comply with the rules at https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html. However, we did not manage to fully implement this, so it is not used in production.

The system uses a reverse proxy, which takes HTTPS requests and decrypts them. This is done with Caddy, as was seen in figure 1. Upon reflection on the system, and the information it exposes, it should have been hidden behind the reverse proxy and had firewalls, as many ports can still be reached with HTTP.

2.5 Availability and scaling

To scale and ensure availability of the application Terraform and Docker Swarm is used.

2.5.1 Availability

The application consists of five nodes that are a part of a swarm cluster, 2 workers, 2 managers and 1 leader, creating redundancy, that, together, run four replicated MiniTwit services. All nodes run Alloy. The “Leader” node also runs Caddy, Grafana, Prometheus and Loki. Figure 1 illustrates this. See `terraform/minitwit_swarm_cluster.tf` and `docker_swarm/stack/minitwit_stack.yml` under `/infrastructure` for the code. Having multiple services running MiniTwit ensures that even if a docker service or swarm node fails, the application is still available.

Single Point of Failure: Multiple services like Caddy and Grafana are pinned to the “Leader” node. This creates SPOF for those services, e.g. only having one Caddy load balancer service, living on the “Leader” node, that takes HTTPS requests on our `devbobs.tech` domain. So, if the “Leader” fails, HTTPS request would no longer be recieved by the system. If the “Leader” goes down, it is still possible to access the other nodes IPs on port :8080 without HTTPS. The docker ingress routing mesh will take care of load balancing, instead of Caddy.

2.5.2 Scaling

The application can be scaled in two different ways: **Vertical scaling:** A single node can be scaled vertically either by resizing the node in DigitalOcean or changing the `size` field in `minitwit_swarm_cluster.tf` and applying it with Terraform. **Horizontal scaling:** The deployment of the application can be scaled horizontally by increasing the amount of nodes in the cluster. This can be done by increasing the `count` number of e.g. a Worker node and then running the `bootstrap.sh` script again, which applies the change with Terraform.

Horizontal scaling can also be achieved by increasing the amount of MiniTwit instances per node, increasing redundancy. When scaling horizontally the load balancer distributes the requests.

3 Reflection Perspective

This section will focus on our reflections about the process, some of our major issues and what we have learned for improving the process in the future.

3.1 Devops-style work

This project has shown how powerful it can be to bridge the gap between development and operations. By automating processes, such as a full CI/CD pipeline, the distance and complexity of going from a feature branch to development have become small and manageable. The project created a short feedback loop, which helped opening up for more continuous work, both in the world of development, Devops and operations. To help enable a short feedback loop, we have tried to keep our work visible. This was done through the use of conventional commits, a developer notebook for progress and changes, a project board, and group chat for emergencies. This also allowed for showing changes and thereby fueling continual, organisational learning when local discoveries occurred. An example of an issue in our project board can be seen here https://github.com/DevBobs-devops/Devbobs_MiniTwit/pull/66.

As the project grew it became harder to diagnose and understand why something failed. With the monitoring and logging setup this became easier, as everything was collected in one place and allowed for quicker feedback. In retrospect, our setup could have been better, which mostly comes down to refining the dashboards. We will touch more on this in 3.2.2.

3.2 Biggest challenges during development

This section will touch on some of the biggest issues we had with regards to evolution, and refactoring and operation, and maintenance.

3.2.1 Refactoring and evolution

The project is based on our old Chirp project with a sqlite database. This gave us some difficulties when migrating database, as data in the old database did not fit the new Postgres database. Upon reflection, using Postgres from the beginning would have been better if we knew the project would grow big, as the Sqlite database would not be sufficient for a proper system. This we could not have known at the time we built Chirp, but shows how evolution of a system will require refactoring later if such evolution was not thought of when building the system. As the project went on, we updated our old source code to adapt to the new changes when relevant, attempting to remove possible errors caused by our old source code. An example of outdated legacy code, that is still being used, is our docker-compose file. It is only used in our CI for testing, whereas in production we use our new stack file, that deploys our docker swarm.

3.2.2 Logging

Logging has been a constant issue. Logs would “disappear” after a while using Grafana log drilldown, only showing logs for one or two of our containers. We instead made a log dashboard to show the logs, like the MiniTwit dashboard, fixing the problem.

As a consequence of these problems with drilldown, we often had to check the logs on the server, we would ssh into it, instead of using the grafana dashboard. Having more logs show up in grafana, for example more statistics about queries to the database, could have given us more information about the more niche parts of our program and errors.

3.2.3 Test-server

Having a test-server would have been beneficial for this project, as we currently only have our “main” server. This would have allowed us to further test new features or fixes, before they ended up in production. This also holds true for the database, as we often had to be very cautious out of fear that we would overwrite or delete data. We did make another DigitalOcean project near the end of the project, to test Terraform and docker swarm, however this never got added to our pipeline.

4 Use of Generative AI

Generative AI has primarily been used for debugging errors and understanding new tools. It was also used for code assistance, especially with new syntax that was unfamiliar to us. We have used ChatGPT, Claude AI and Microsoft Copilot. They have been co-authored when they have been used, often with a small message, explaining what they helped with. AI has also been used to catch potential errors by e.g. confirming syntax before running a new GitHub workflow. Obviously, there is a possibility of loss of learning, as AI speeds up the problem-solving process, sometimes highlighting issues we had not observed yet. When writing this report, we discovered that we wrote Co-authors of the AI's in a wrong format - missing an email. We could therefore not include them in the mailmap. Instead they can be found by going through the gitlog and grep'ing Claude and ChatGPT. The total number of commits co-authored with AI: 31.

5 Collection of links to artifacts

The GitHub repository:

1. https://github.com/DevBobs-devops/Devbobs_MiniTwit

The deployed system can be found at:

1. <https://devbobs.tech/>

The monitoring and logging dashboard can be found at:

1. <http://134.199.190.79:3000/dashboards>

The project board can be found at:

1. <https://github.com/orgs/DevBobs-devops/projects/2>

The various docker images can be found on DockerHub:

1. <https://hub.docker.com/u/powsdaws>

A Dependencies

NuGet Dependencies

The dependencies of the MiniTwit application, listed using `dotnet list package`, are as follows.

- Microsoft.AspNetCore.DataProtection.EntityFrameworkCore 8.0.* 8.0.26
- Microsoft.AspNetCore.Identity.EntityFrameworkCore 8.0.10 8.0.10
- Microsoft.AspNetCore.Identity.UI 8.0.10 8.0.10
- Microsoft.CodeAnalysis.NetAnalyzers 10.0.201 10.0.201
- Microsoft.EntityFrameworkCore.Design 8.0.10 8.0.10
- Microsoft.EntityFrameworkCore.Sqlite 8.0.10 8.0.10
- Microsoft.EntityFrameworkCore.SqlServer 8.0.10 8.0.10
- Microsoft.EntityFrameworkCore.Tools 8.0.10 8.0.10
- Microsoft.Playwright.NUnit 1.49.0 1.49.0
- Microsoft.VisualStudio.Web.CodeGeneration.Design 8.0.6 8.0.6
- Npgsql.EntityFrameworkCore.PostgreSQL 8.0.4 8.0.4
- prometheus-net 8.2.1 8.2.1
- prometheus-net.AspNetCore 8.2.1 8.2.1
- Swashbuckle.AspNetCore 10.1.2 10.1.2
- coverlet.collector 3.2.0 3.2.0
- Microsoft.AspNetCore.Mvc.Testing 8.0.8 8.0.8
- Microsoft.NET.Test.Sdk 17.8.0 17.8.0
- xunit 2.4.2 2.4.2
- xunit.runner.visualstudio 2.4.5 2.4.5
- NUnit3TestAdapter 4.5.0 4.5.0

Docker Dependencies

The external docker dependencies are as follows.

- mcr.microsoft.com/dotnet/aspnet:8.0
- mcr.microsoft.com/dotnet/sdk:8.0
- grafana/loki:3.6.7
- prom/prometheus:v3.5.1
- grafana/alloy:latest
- grafana/grafana:12.1
- lucaslorentz/caddy-docker-proxy:ci-alpine

The internal dependencies that is created when deploying are as follows.

- powsdaws/minitwitimage
- powsdaws/minitwit-prometheus
- powsdaws/minitwit-alloy
- powsdaws/minitwit-loki
- powsdaws/minitwit-grafana

Github Action dependencies

The dependencies of our CI/CD pipeline workflows are as follows.

- actions/checkout 2.*.*
- actions/checkout 3.*.*
- actions/checkout 4.*.*
- docker/build-push-action 6.*.*
- docker/build-push-action
d08e5c354a6adb9ed34480a06d141179aa583294
- docker/login-action 3.*.*
- docker/setup-buildx-action 3.*.*

- actions/setup-dotnet 4.*.*
- actions/setup-python 5.*.*
- actions/upload-artifact 4.*.*
- guibranco/github-csharpier-linter-action
a5214a476742ce4a0f755611581276c50081527a
- hadolint/hadolint-action
2332a7b74a6de0dda2e2221d575162eba76ba5e5